Hash

João Victor Pereira · Segurança em Sistemas de Computação

1) SHA-256 passo a passo

O SHA-256 pega uma mensagem de qualquer tamanho e gera um resumo de 256 bits (64 caracteres em hexadecimal). Os passos são:

1. **Converter para binário:** a mensagem é transformada em bits (cada caractere vira 8 bits).
2. **Padding:** coloca um bit 1 no final, depois vários zeros, e nos últimos 64 bits coloca o tamanho original da mensagem. Os zeros são o quanto precisar para o total ficar múltiplo de 512 bits.
3. **Dividir em blocos:** a mensagem é dividida em blocos de 512 bits, e cada bloco é processado um de cada vez.
4. **Valores iniciais:** o hash começa com 8 valores fixos de 32 bits (H0 a H7), que vêm das raízes quadradas dos 8 primeiros números primos. Também existem 64 constantes K, que vêm das raízes cúbicas dos 64 primeiros primos.
5. **Message schedule:** cada bloco é quebrado em 16 palavras de 32 bits, e a partir delas são calculadas mais 48, totalizando 64 palavras (w0 a w63). As novas palavras são feitas com rotações, deslocamentos, XOR e somas das anteriores (funções σ0 e σ1).
6. **Compressão:** copia H0..H7 para as variáveis a, b, c, d, e, f, g, h e faz 64 rodadas. Em cada rodada calcula dois valores temporários usando as funções Σ0, Σ1, Choice (escolhe bits de f ou g dependendo de e) e Majority (pega o bit que aparece mais em a, b e c), junto com a palavra w e a constante K da rodada. Depois as variáveis "andam" uma posição (h recebe g, g recebe f, ...) e a e e recebem os novos valores.
7. **Atualizar o hash:** no fim das 64 rodadas, soma a..h nos valores H0..H7 (módulo 2^{32}). Se tiver mais blocos, repete os passos 5 a 7 usando esses novos H.
8. **Resultado:** junta H0 até H7 e esse é o hash final. Por exemplo, o SHA-256 de `hello world` é `b94d27b9934d3e08a52e52d7da7dabfac484efe37a5380ee9088f7ace2efcde9`.

2) Principais algoritmos de hash usados hoje

Hoje os algoritmos de hash mais conhecidos são o MD5, o SHA-1, a família SHA-2, o SHA-3 e o BLAKE2/BLAKE3. Também tem os hashes feitos para senha, como bcrypt e Argon2.

O MD5 e o SHA-1 são rápidos e ainda aparecem bastante em sistemas antigos e para conferir se um arquivo baixou certo, mas os dois já estão quebrados para segurança, porque dá pra gerar colisões (duas entradas com o mesmo hash). No SHA-1 isso foi mostrado na prática em 2017 com o ataque SHAttered, então não devem ser usados para assinatura nem certificado.

O SHA-2 (principalmente o SHA-256 e o SHA-512) é o mais usado hoje, está em certificados TLS, no Git mais novo, no Bitcoin etc. O ponto forte é que não tem ataque prático conhecido e é suportado em tudo. O ponto fraco é que ele usa a mesma estrutura do MD5 e do SHA-1

(Merkle-Damgård), que permite o ataque de extensão de tamanho se for usado de forma errada, por isso para autenticar mensagem se usa HMAC.

O SHA-3 foi criado com uma estrutura diferente (esponja), então não sofre extensão de tamanho e serve de alternativa caso achem problema no SHA-2. O lado ruim é que é mais lento em software e menos usado. O BLAKE2 e o BLAKE3 são bem rápidos e seguros, mas não são padrão do NIST, então aparecem menos.

Por fim, para guardar senha não se deve usar nenhum desses direto, porque eles são rápidos e isso facilita força bruta. Para isso existem o bcrypt e o Argon2, que são lentos de propósito e usam salt. O ponto fraco é justamente gastar mais tempo e memória do servidor.

3) Hash simples com XOR

Fiz em Java dois hashes de 32 bits (arquivos `HashSimples.java` e `Main.java`). O primeiro só faz XOR de todos os blocos de 4 bytes. O segundo rotaciona o hash 5 bits antes de cada XOR e no final mistura o tamanho da mensagem. Saída do programa:

```
"ola mundo"    XOR: 6d190f44   XOR+rotação: 62a88542
"ola mundO"    XOR: 4d190f44   XOR+rotação: 62a88546

Trocando a ordem dos blocos:
"abcdefgh"     XOR: 0404040c   XOR+rotação: 0e7d759b
"efghabcd"     XOR: 0404040c   XOR+rotação: 9eedc40b

Bloco repetido:
"abcdabcd"     XOR: 00000000   XOR+rotação: 8efdf41b
""             XOR: 00000000   XOR+rotação: 468acf02
```

Dá pra ver que o XOR puro é bem fraco: trocar a ordem dos blocos dá o mesmo hash, e um bloco repetido se anula (fica igual ao hash da string vazia). Com a rotação esses dois casos param de colidir, mas ainda não é seguro: mudar uma letra muda só um bit do hash, enquanto num hash de verdade como o SHA-256 metade dos bits mudaria.